

Quick Start

This guide covers the fastest path to getting **Genesys Cloud Auditor** running locally for desktop use or non-interactive runner execution.

Use this document for:

- local setup
- OAuth credential configuration
- app settings
- build and run steps
- workbook output validation
- basic troubleshooting

For product direction and planned capabilities, see [ROADMAP.md](#).

For the project overview, see [README.md](#).

Prerequisites

Before running the application, make sure you have:

- **Windows 10 or Windows 11** (64-bit)
- **.NET SDK 8.x**
- access to a **Genesys Cloud org**
- a **Genesys Cloud OAuth client** with permissions appropriate to the audit scope
- outbound HTTPS access to the correct regional login and API endpoints

Recommended:

- PowerShell 7+
 - Excel installed locally if you want to inspect workbook output comfortably
 - a test or lower-risk tenant for first execution
-

OAuth Requirements

Genesys Cloud Auditor needs a valid OAuth client capable of calling the APIs used by the audit modules.

At minimum, confirm you have:

- a valid **Client ID**
- a valid **Client Secret** if using client-credential style auth
- the correct **Genesys Cloud region**
- permissions/scopes sufficient to read the users, routing, telephony, architect, and other relevant audit domains

If your org uses centrally managed OAuth provisioning, confirm the application registration details before troubleshooting the code. Many “app bugs” are really auth gremlins in a trench coat.

1. Clone the Repository

```
git clone <YOUR_REPOSITORY_URL>
cd GenesysCloudAuditor
```

2. Restore Dependencies

```
dotnet restore
```

3. Configure Secrets for Local Development

Use .NET user-secrets for local development rather than hard-coding credentials.

Desktop app secrets

```
cd src\GenesysExtensionAudit.App
dotnet user-secrets init
dotnet user-secrets set "GenesysOAuth:ClientId" "YOUR_CLIENT_ID"
dotnet user-secrets set "GenesysOAuth:ClientSecret" "YOUR_CLIENT_SECRET"
cd ..\..
```

Runner secrets

If the runner uses a separate project configuration scope, set secrets there too:

```
cd src\GenesysExtensionAudit.Runner
dotnet user-secrets init
dotnet user-secrets set "GenesysOAuth:ClientId" "YOUR_CLIENT_ID"
dotnet user-secrets set "GenesysOAuth:ClientSecret" "YOUR_CLIENT_SECRET"
cd ..\..
```

If your solution centralizes configuration through shared settings, adapt this accordingly.

4. Configure App Settings

Edit the appropriate `appsettings.json` file for your environment.

Typical files:

- `src/GenesysExtensionAudit.App/appsettings.json`
- `src/GenesysExtensionAudit.Runner/appsettings.json`

Example

```
{
  "Genesys": {
    "Region": "mypurecloud.com",
    "PageSize": 100,
    "IncludeInactive": false,
    "MaxRequestsPerSecond": 3
  },
  "Audit": {
    "EnableAuditLogs": true,
    "EnableOperationalEvents": true,
    "EnableOutboundEvents": true
  },
  "Export": {
    "OutputDirectory": "C:\\\\AuditExports"
  }
}
```

Key settings to verify

Genesys:Region

Examples may include:

- `mypurecloud.com`
- `usw2.pure.cloud`
- other region-specific values depending on your tenant

This must match the org you are auditing.

Genesys:PageSize

Controls collection page sizes for API retrieval.

Larger values may reduce calls but can stress memory or amplify retry pain if something goes sideways.

Genesys:IncludeInactive

Determines whether inactive users are included in certain audit paths.

Genesys:MaxRequestsPerSecond

Use a conservative starting value.

It is better to be slightly slower than to get smacked repeatedly with 429 responses like an impatient API

goblin.

Audit:*

Enable or disable event-heavy collectors and optional audit paths.

Export:OutputDirectory

Set this to a writable folder that exists or can be created by the app.

5. Build the Solution

```
dotnet build -c Release
```

If the build fails:

- confirm the .NET SDK version
 - confirm all referenced projects restore successfully
 - review any environment-specific paths or SDK assumptions in the solution
-

6. Run the Desktop Application

```
dotnet run --project GenesysExtensionAudit.App.csproj
```

Expected desktop behavior

The desktop app should allow you to:

- review configuration
- initiate an audit
- monitor progress
- inspect findings
- export a workbook

If the UI starts but no data is returned, check auth, region, permissions, and throttling before blaming the poor innocent window.

7. Run the Headless Runner

Use the runner for scheduled or unattended execution.

Example:

```
dotnet run --project
src\GenesysExtensionAudit.Runner\GenesysExtensionAudit.Runner.csproj -- --dry-
run
```

Then execute a real run with your intended parameters.

If your runner supports additional arguments such as output path, schedule profile, SharePoint upload, or specific audit scopes, document those in a later [docs/Operations.md](#) file.

8. Perform Your First Audit

Recommended first pass:

- use a non-production or lower-risk org if available
- keep optional collectors enabled only if you need them
- start with default paging and conservative rate settings
- export results after the first successful run
- inspect workbook sheet names and content before scaling usage

Suggested first-run checklist

- OAuth auth succeeds
 - region resolves correctly
 - at least one user/queue/flow-related sheet is populated
 - workbook export completes successfully
 - no unexpected fatal errors in logs
 - output folder contains the expected file(s)
-

9. Review Outputs

The application is expected to produce a workbook containing findings and supporting data exports.

Typical outputs may include:

- extension integrity sheets
- DID mismatch sheets
- queue and group hygiene sheets
- stale flow sheets
- audit log export sheets
- operational event sheets
- outbound event sheets
- summary-oriented sheets if implemented

What to validate after a run

- workbook was created

- sheet names match expected audit areas
- row counts are plausible
- known tenant issues appear where expected
- empty sheets are explainable rather than suspicious
- timestamps and file names align with the run you just executed

A workbook with zero useful data is either a sign of a pristine org or a lying machine. Statistically, the second creature appears surprisingly often.

10. Run Tests

```
dotnet test tests\GenesysExtensionAudit.Infrastructure.Tests\
```

Recommended test categories over time:

- collector tests
 - normalization tests
 - correlation tests
 - export tests
 - snapshot/diff tests
 - care-packet generation tests
-

11. Common Troubleshooting

Authentication Issues

Symptom

401 Unauthorized

Likely causes

- invalid client ID or client secret
- wrong region
- token request misconfiguration
- secret not loaded into the active project scope

Actions

- verify the configured OAuth client
 - verify secrets were set for the correct project
 - verify the region value
 - verify the app is actually reading the intended configuration source
-

Authorization Issues

Symptom

403 Forbidden

Likely causes

- OAuth client lacks required permissions
- admin API visibility restrictions
- tenant policy or division scoping prevents access

Actions

- compare required endpoints with assigned permissions
 - validate the client has read access to the audited domains
 - review org-specific access constraints
-

Region Mismatch

Symptom

Login works poorly, API calls fail unexpectedly, or endpoints appear unavailable.

Likely causes

- **Region** value does not match the tenant's actual environment

Actions

- confirm the tenant region
 - align login and API endpoint assumptions
 - retest with corrected region setting
-

Throttling / Rate Limiting

Symptom

429 Too Many Requests

Likely causes

- request rate too high
- event-heavy collection across large tenants
- insufficient backoff/retry tuning

Actions

- lower **MaxRequestsPerSecond**
 - reduce optional collectors during testing
-

- run during lower activity windows
 - inspect retry behavior
-

Slow Runs

Symptom

Audit completes very slowly on large orgs

Likely causes

- conservative throttling
- large tenant size
- event-heavy collectors enabled
- expensive endpoint combinations

Actions

- confirm current page size and request rate
 - disable optional collectors temporarily
 - test narrower audit scopes if supported
 - compare desktop and runner behavior
-

Empty or Suspiciously Small Results

Symptom

Workbook generates but contains little or no useful data

Likely causes

- auth succeeded but visibility is limited
- filters excluded expected data
- inactive users omitted
- optional collectors disabled
- collection failure handled too quietly

Actions

- review effective runtime settings
 - check logs
 - validate permissions
 - test with `IncludeInactive = true` when appropriate
 - inspect whether collectors are enabled
-

Export Problems

Symptom

Workbook does not write successfully

Likely causes

- invalid or unwritable output path
- file locked by Excel
- missing export dependency
- permissions issue on destination folder

Actions

- close the workbook if already open
- use a simple local output directory
- confirm the folder exists and is writable
- rerun and inspect logs

12. Scheduling Recurring Runs

For recurring health checks, use the headless runner with Windows Task Scheduler or your preferred scheduling/orchestration platform.

Good recurring use cases include:

- nightly tenant hygiene audits
- pre-change / post-change comparisons
- weekly configuration integrity checks
- support evidence collection after incidents
- baseline snapshot generation

Recommended later documentation split:

- keep this file focused on getting started
- document full scheduled-run patterns in [docs/Operations.md](#)

13. Recommended First Operational Pattern

A practical early pattern for real usage:

Daily or weekly runner execution

Generate a workbook on a schedule.

Save output to a consistent directory

Example:

Preserve historical files

Do not overwrite everything immediately. Historical comparisons become very valuable once the app grows drift intelligence.

Review findings in three buckets

- obvious tenant configuration fixes
- changes that need review by internal teams
- contradictions that may justify Genesys Care escalation

This is the bridge from “audit tool” to “evidence workbench.”

14. Next Documents to Read

After completing initial setup, the most useful next documents are:

- [README.md](#)
- [ROADMAP.md](#)
- [docs/Configuration.md](#)
- [docs/Operations.md](#)
- [docs/Audit-Checks.md](#)
- [docs/Troubleshooting.md](#)

If those docs do not exist yet, that is not a crisis. It just means the doc ecosystem is still evolving and has not fully molted into its final form.

15. Quick Validation Checklist

Use this after initial setup:

- ☐ repository cloned successfully
 - ☐ dependencies restored
 - ☐ OAuth secrets configured
 - ☐ region verified
 - ☐ appsettings reviewed
 - ☐ solution builds successfully
 - ☐ desktop app launches
 - ☐ first audit completes
 - ☐ workbook export succeeds
 - ☐ test suite runs
 - ☐ logs show no unexplained fatal errors
-

Notes

This Quick Start intentionally focuses on the shortest path to successful execution.

It does not yet attempt to fully document:

- detailed endpoint permissions
- complete configuration reference
- audit rule internals
- historical snapshot behavior
- support case packet structure
- advanced scheduling / automation patterns

Those belong in the deeper docs as the project matures.